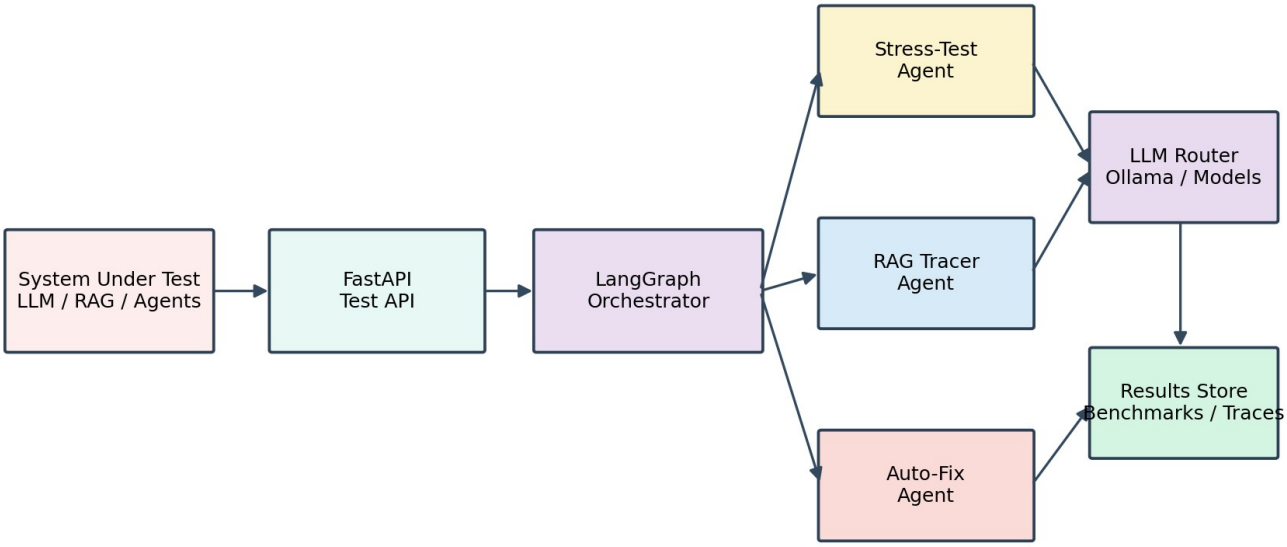
AgentProbe

AI Evaluation and Regression Platform

Engineering Case Study

AgentProbe High-Level Architecture



Author	Venkata Lakshmi Parimala Pasupuleti
Project Type	Portfolio AI engineering system
Primary Purpose	Pre-deployment testing, benchmarking, and debugging of LLM, RAG, and multi-agent systems
Core Stack	Python, LangChain, LangGraph, FastAPI, ChromaDB, Ollama, React
Execution Model	Local-first; designed to run without paid model APIs
Document Status	Technical case study; implementation claims separated from production targets

Document Map

1. Executive Summary
2. Problem and Motivation
3. Scope and Status Boundaries
4. System Architecture
5. Adaptive Orchestrator Agent
6. RAG Tracer Agent
7. Auto-Fix Agent
8. LLM Router and Local Execution
9. Test Input and Benchmark Model
10. Evaluation Dimensions
11. Failure Taxonomy
12. Trace and Evidence Model
13. API and User Interface
14. Data and Storage Design
15. Reliability and Safety
16. Performance and Scalability
17. Security and Privacy
18. CI/CD Regression Gate
19. Observability and Operations
20. Design Decisions and Trade-offs
21. Known Limitations
22. Modernization Roadmap
23. Lessons Learned
24. Capability Matrix and Conclusion

NOTE	The case study is grounded in the documented project description. Detailed production extensions are explicitly labeled as TARGET rather than represented as implemented.

1. Executive Summary

AgentProbe is a pre-deployment testing, benchmarking, and debugging toolkit for LLM applications, retrieval-augmented generation pipelines, and multi-agent workflows. The project addresses a practical gap: traditional software teams have unit tests, integration tests, CI pipelines, and release gates, while AI applications often rely on manually inspecting a small number of outputs and assuming that prompt or model changes improved quality.

The platform uses three specialized LangGraph agents. The Orchestrator Agent runs an adaptive three-phase stress-test cycle: it first explores broad behavior, then concentrates tests on weak areas, and finally verifies that discovered failures are reproducible. The RAG Tracer Agent examines a seven-stage retrieval pipeline to identify the stage where quality degraded. The Auto-Fix Agent generates concrete remediation proposals such as prompt rewrites, chunking adjustments, and query rewrites.

A model router selects an appropriate local model for each task, and the documented implementation is designed to run locally through Ollama, avoiding paid API dependency. FastAPI provides the service layer, React provides the user interface, LangGraph coordinates agent state and transitions, and ChromaDB supports vector-oriented test and diagnostic data.

BUILT	Three LangGraph agents, adaptive stress-testing flow, RAG tracing, generated fix recommendations, local LLM routing, FastAPI service layer, React interface, and a local-first execution model are part of the documented project implementation.

TARGET	Enterprise CI enforcement, organization-wide benchmark governance, production telemetry integration, distributed workers, and policy-managed release gates are modernization targets.

2. Problem and Motivation

LLM systems fail differently from deterministic applications. A conventional function usually produces the same output for the same input, while a model response can change because of prompt wording, retrieved context, model version, sampling parameters, tool behavior, or upstream data. A change that improves one scenario may silently damage another.

Operational problems AgentProbe addresses

- Prompt changes are often evaluated with a few hand-picked examples rather than a representative benchmark.
- RAG failures are frequently attributed to the final model even when the actual defect occurred during query rewriting, retrieval, filtering, reranking, or context construction.
- Multi-agent workflows can reach a plausible final answer while following an incorrect or inefficient trajectory.
- Quality regressions are difficult to detect before deployment when teams lack baseline comparison and repeatable test suites.
- Debugging is slow when traces, retrieved evidence, intermediate prompts, and model responses are not captured together.
- Model selection is often static even though different evaluation tasks have different latency, reasoning, and cost needs.

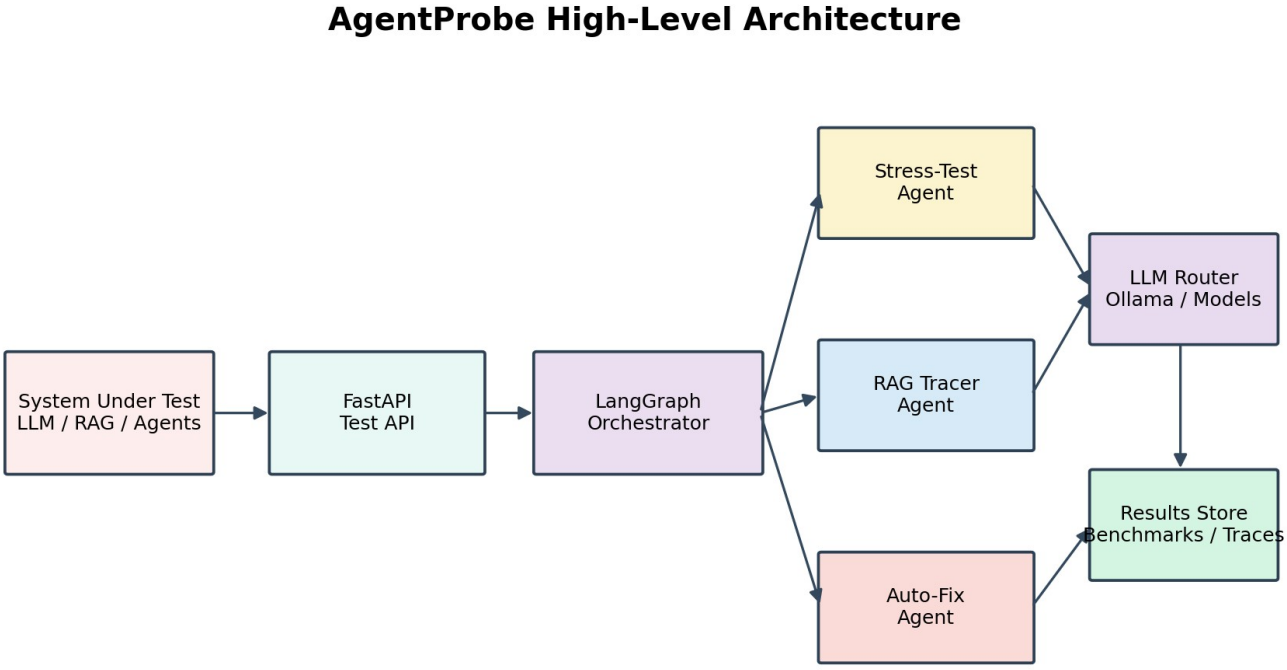
AgentProbe reframes AI quality as an engineering discipline. The objective is not to prove that a model is generally intelligent. The objective is to determine whether a specific AI system meets a defined behavioral contract across a controlled suite of tests, and to provide actionable evidence when it does not.

3. Scope and Status Boundaries

Capability	Status	What is supported
Adaptive three-phase stress testing	BUILT	Explore broad behavior, target weak spots, and verify discovered failures.
Seven-stage RAG tracing	BUILT	Diagnose where retrieval quality or context construction degraded.
Generated remediation recommendations	BUILT	Produce prompt rewrites, chunk configuration changes, and query rewrites.
LLM routing	BUILT	Choose a model appropriate to the task.
Local execution	BUILT	Run with Ollama to avoid paid model APIs.
FastAPI and React application	BUILT	Expose testing workflows through an API and user interface.
ChromaDB integration	BUILT / PROJECT STACK	Vector-oriented storage is part of the documented stack.
CI/CD policy enforcement	TARGET	Automatically block release when quality policies fail.
Distributed benchmark execution	TARGET	Run large suites through queues and multiple workers.
Production telemetry feedback loop	TARGET	Turn real production failures into regression cases.

The platform is a portfolio engineering project rather than a production deployment inside a company. The document therefore distinguishes between implemented product behavior and the operational capabilities that would be required for enterprise adoption.

4. System Architecture



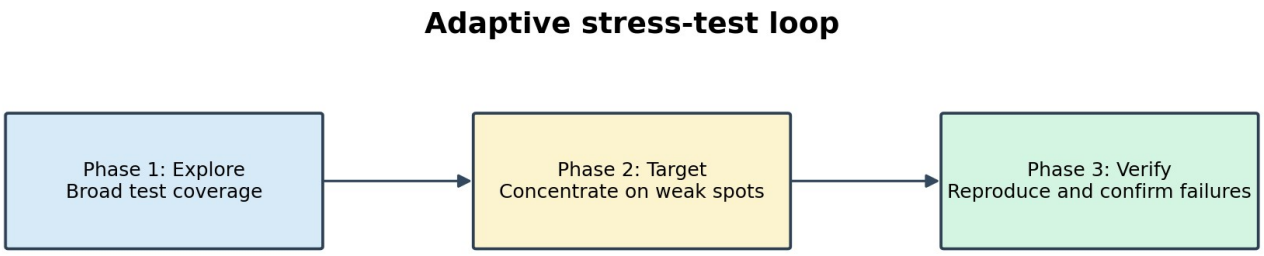
The system under test may be a direct LLM call, a RAG pipeline, or a multi-agent workflow. FastAPI receives the test configuration and delegates execution to a LangGraph coordinator. The graph maintains test state, selects an evaluation path, invokes specialized agents, collects traces, and produces a structured report.

Component	Responsibility
FastAPI service	Accept test definitions, launch evaluations, expose status and results.
LangGraph orchestrator	Maintain state, route evaluation stages, and manage iterative testing.
Orchestrator Agent	Generate and adapt stress-test cases.
RAG Tracer Agent	Inspect seven retrieval and generation stages.
Auto-Fix Agent	Transform observed failures into concrete remediation proposals.
LLM Router	Select the most suitable local model for each evaluation task.
ChromaDB / result storage	Persist evaluation artifacts, vectors, traces, and benchmark results.
React UI	Configure tests and inspect reports, traces, and recommendations.

Architectural principle

Testing logic is separated from the system being tested. AgentProbe observes inputs, outputs, intermediate traces, and benchmark expectations without becoming part of the production decision path of the target application. This keeps the evaluator replaceable and reduces the risk that test logic changes production behavior.

5. Adaptive Orchestrator Agent



Phase 1 - Explore

The initial phase creates broad coverage across known capabilities, input styles, edge cases, and failure categories. Its purpose is to establish the system's baseline behavior and identify areas where failures cluster.

Phase 2 - Target weak spots

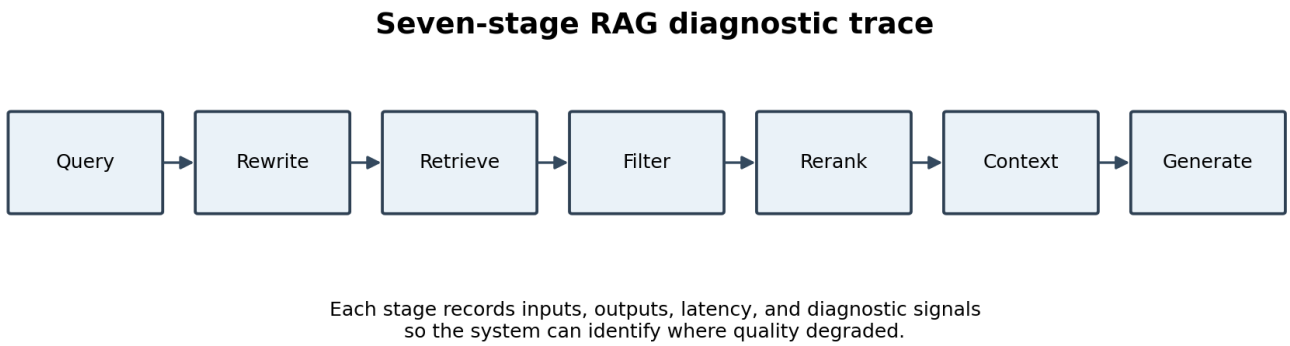
The orchestrator adapts subsequent tests based on the first-pass evidence. Instead of repeatedly sampling already-successful paths, it concentrates on fragile prompts, ambiguous queries, weak retrieval topics, inconsistent agent transitions, or scenarios with low-quality outputs.

Phase 3 - Verify failures

A suspected defect is not treated as confirmed after one surprising response. The verification phase reruns or mutates the failing scenario to determine whether the behavior is reproducible, input-specific, or caused by normal model variance.

VALUE	This three-phase loop uses evaluation budget more effectively than a fixed suite alone because it spends additional effort where evidence suggests the system is weakest.

6. RAG Tracer Agent



A RAG answer can be wrong even when the generator model behaves correctly. The RAG Tracer therefore decomposes the pipeline into seven diagnostic stages. The documented project description identifies seven pipeline steps; the diagram uses a representative sequence to make the debugging model explicit.

Stage	Primary diagnostic question
Query	Is the original user intent captured accurately?
Rewrite	Did reformulation preserve important entities and constraints?
Retrieve	Were relevant documents or chunks returned?
Filter	Did metadata or policy filtering remove required evidence?
Rerank	Were the strongest candidates promoted?
Context	Was the final context complete, ordered, and within limits?
Generate	Did the model remain grounded in the supplied evidence?

The tracer's purpose is fault localization. Rather than returning only "the answer was wrong," it identifies the earliest stage at which the pipeline diverged from expected behavior. This reduces debugging time and avoids rewriting prompts when the actual problem is retrieval configuration.

7. Auto-Fix Agent

The Auto-Fix Agent converts diagnostic evidence into concrete remediation proposals. The documented implementation produces rewritten prompts, revised chunk configurations, and query rewrites. These recommendations are generated as candidate fixes, not silently applied changes.

Failure evidence	Candidate response
The model ignores an output constraint	Rewrite the system or task prompt with explicit schema and rejection behavior.
Relevant evidence is split across chunks	Increase or restructure chunk overlap; align chunk boundaries with document sections.
Queries use vocabulary different from the corpus	Generate query expansions or a domain-specific rewrite.
Retriever returns broad but weak matches	Tighten filters, adjust top-k, or introduce reranking.
Context contains contradictory versions	Add version metadata and precedence rules before generation.

Safety boundary

Recommendations should be reviewed and rerun through the benchmark before they are accepted. Automatically applying a generated fix would bypass the very regression discipline the platform is intended to introduce.

8. LLM Router and Local Execution

The model router chooses an appropriate model for each task rather than sending every workload to one model. Evaluation workloads vary: some require structured classification, some require long-context analysis, some require critique, and others require concise rewrite generation.

- Use a lightweight model for deterministic formatting or simple classification.
- Use a stronger reasoning model for failure analysis and remediation generation.
- Prefer local Ollama models when privacy, repeatability, or zero-cost execution is the priority.
- Keep model selection explicit in the trace so results remain reproducible and explainable.

The project is documented as running 100% locally at zero cost. This is meaningful for experimentation because it removes API expense from repeated stress tests and allows test data to remain on the user's machine. The trade-off is that local model quality and throughput depend on available hardware.

BUILT	Local-first model execution and smart task-based routing are documented capabilities.

9. Test Input and Benchmark Model

An evaluation platform needs structured test definitions. The documented project confirms benchmarking and adaptive stress tests, but does not specify one fixed schema. A practical benchmark record should contain the following information.

Field	Purpose
case_id	Stable identity used for baseline comparison.
system_type	LLM, RAG, or multi-agent workflow.
input	Prompt, query, document reference, or workflow request.
expected_behavior	Required outcome, constraint, or rubric.
reference_evidence	Ground truth or expected source material when available.
tags	Domain, capability, severity, and failure category.
execution_policy	Model, temperature, retry count, and timeout.
acceptance_policy	Pass thresholds and blocking conditions.

This schema is presented as a production-oriented extension. The core project capability is the ability to execute and benchmark systems; benchmark governance and organization-wide test ownership are targets for a mature implementation.

10. Evaluation Dimensions

AI quality cannot be represented by one score. AgentProbe should treat evaluation as a collection of complementary dimensions. The following categories provide an engineering model for interpreting test outcomes.

Dimension	Examples
Task success	Correct classification, correct answer, required tool completion.
Grounding	Claims supported by retrieved or supplied evidence.
Retrieval quality	Relevant evidence found and irrelevant evidence minimized.
Trajectory quality	Expected agents, tools, and transitions used.
Constraint adherence	Schema, tone, safety, and business-policy compliance.
Consistency	Repeated trials produce acceptably stable behavior.
Latency	End-to-end and per-stage response time.
Resource efficiency	Token usage, model utilization, and local compute cost.
Actionability	Diagnostic reports and generated fixes can be used by an engineer.

The documented project emphasizes testing, benchmarking, and debugging but does not provide a published metric formula or production threshold set. Thresholds therefore belong to the target governance model and should be established per application and risk level.

11. Failure Taxonomy

Failure class	Description	Primary owner
Prompt defect	Instructions are ambiguous, conflicting, or incomplete.	Prompt / application engineer
Retrieval miss	Relevant evidence is not returned.	Search / RAG engineer
Context defect	Correct evidence is truncated, disordered, or contradicted.	RAG pipeline owner
Model reasoning defect	Evidence is present but conclusion is incorrect.	Model / application owner
Tool failure	Tool is not selected, receives invalid arguments, or returns an error.	Integration owner
Agent trajectory defect	Workflow reaches an invalid state or skips required checks.	Agent platform owner
Safety defect	System produces prohibited or unsupported behavior.	Product and safety owner
Performance regression	Latency or resource use exceeds policy.	Platform owner

A consistent taxonomy allows the platform to aggregate failures across test runs and direct work to the appropriate engineering layer. It also prevents every AI defect from being labeled generically as a “model problem.”

12. Trace and Evidence Model

Debugging requires a complete chain of evidence. Each run should preserve the original input, selected model, prompts, retrieval results, agent transitions, tool inputs and outputs, generated response, evaluator findings, and proposed remediation.

Artifact	Why it matters
Run metadata	Reproduce the environment and configuration.
Prompt versions	Determine whether wording changes caused regression.
Retrieved chunks	Verify evidence quality and rank order.
Agent state transitions	Inspect workflow correctness.
Tool calls	Validate tool selection and arguments.
Model outputs	Compare raw behavior before post-processing.
Scores and findings	Explain why a run passed or failed.
Fix recommendations	Connect diagnosis to remediation.

For production use, sensitive inputs should be redacted or access-controlled. Full traces are valuable for debugging but can contain proprietary prompts, customer data, or retrieved content.

13. API and User Interface

FastAPI forms the service boundary and React provides the documented frontend. The exact endpoint contract is not specified in the available project description; the following interface represents a production-ready API design consistent with the implementation.

Endpoint	Purpose
POST /test-runs	Launch a benchmark or adaptive stress test.
GET /test-runs/{id}	Retrieve status, summary, and policy outcome.
GET /test-runs/{id}/traces	Inspect stage-level traces and artifacts.
POST /test-runs/{id}/verify	Rerun a suspected failure for reproducibility.
GET /test-runs/{id}/fixes	Return generated remediation proposals.
GET /benchmarks	List benchmark suites and baseline versions.

Frontend views

- Test configuration and target-system registration.
- Benchmark dashboard with pass/fail summaries and trends.
- RAG pipeline trace visualization.
- Failure detail page with evidence and stage attribution.
- Auto-fix review and side-by-side comparison.
- Local model routing and runtime configuration.

STATUS	FastAPI and React are documented implementation technologies. The detailed endpoint and screen breakdown above is a design specification, not a claim about exact existing routes.

14. Data and Storage Design

ChromaDB is part of the documented technology stack. A testing platform also needs durable structured storage for runs, cases, scores, and artifacts. The following model separates semantic content from transactional evaluation records.

Store	Data
ChromaDB	Embeddings for test cases, failure examples, traces, and similarity-based diagnosis.
Relational or document store - TARGET	Benchmarks, runs, configurations, scores, model versions, and policy outcomes.
Object storage - TARGET	Large traces, exported reports, documents, and attachments.
Local filesystem - DEMO	Simple local artifacts and project-level execution data.

Entity	Representative fields
benchmark_suite	id, name, version, owner, risk_class
test_case	id, suite_id, input, expected_behavior, tags
test_run	id, suite_version, target_version, started_at, status
stage_trace	run_id, stage, input_ref, output_ref, latency
finding	run_id, category, severity, evidence_ref
fix_candidate	finding_id, change_type, proposed_value, verification_status

15. Reliability and Safety

Reproducibility controls

- Record model name and version, prompt version, sampling configuration, and routing decision.
- Use repeated trials for stochastic scenarios rather than trusting one run.
- Preserve baseline results and benchmark versions.
- Distinguish infrastructure errors from model-quality failures.
- Verify suspected failures before promoting them to blocking regressions.

Auto-fix controls

- Generated fixes remain proposals until reviewed.
- Every proposed fix must be rerun against the full benchmark, not only the failing case.
- Rollback to the previous prompt or retrieval configuration must remain possible.
- The evaluator should not silently modify the target system.

The testing platform itself must avoid becoming a new source of false confidence. An evaluator can be wrong because its rubric is weak, its reference answer is incomplete, or its judge model is biased. Human review remains necessary for benchmark design and high-impact release decisions.

16. Performance and Scalability

Adaptive evaluation can be compute-intensive because it generates new tests, executes the target system, analyzes traces, proposes fixes, and verifies failures. Local execution removes API cost but does not remove resource constraints.

Scaling concern	Design response
Large benchmark suites	Partition by capability and run independent cases in parallel.
Local model contention	Limit concurrency based on GPU or memory capacity.
Long traces	Store large artifacts separately and load them on demand.
Repeated verification	Cache immutable retrieval inputs while preserving model-run independence.
Multiple target systems	Isolate run configuration and resource quotas by project.
Enterprise scale - TARGET	Introduce a queue, worker pool, and distributed result store.

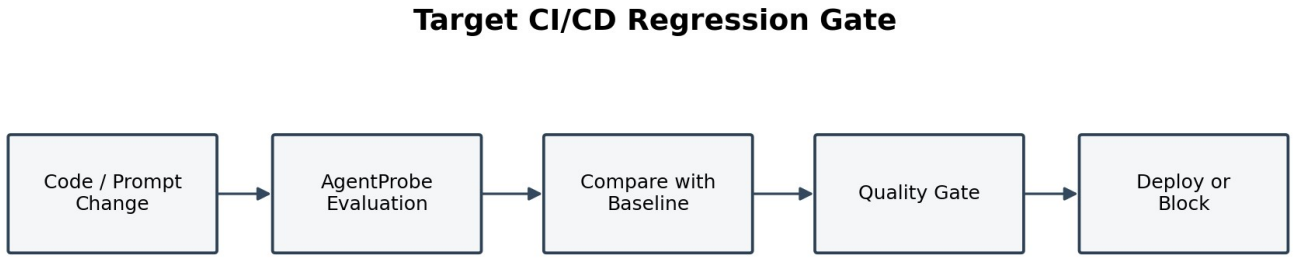
For the documented portfolio scope, local execution is a deliberate simplification. Distributed orchestration is unnecessary until benchmark size, concurrency, or organizational usage justifies the operational cost.

17. Security and Privacy

- Local Ollama execution can keep proprietary prompts and test data off external model APIs.
- Authentication and role-based access are required before multiple teams share benchmarks.
- Traces should be treated as sensitive because they may contain source documents, tool outputs, or customer context.
- Secrets must never be embedded in benchmark files or captured in plain-text traces.
- Exported reports should support redaction and retention policies.
- Auto-fix capabilities should have no direct permission to modify production environments.

The local-first model is an architectural advantage for privacy-sensitive testing. However, local execution alone is not a complete security model. Access control, encryption, secrets management, audit logging, and data lifecycle policies remain necessary for enterprise deployment.

18. CI/CD Regression Gate



Production extension: fail the pipeline when quality, latency, or cost regress beyond policy thresholds.

The strongest production extension is to make evaluation part of the release process. Prompt, retrieval, model, tool, or agent changes should run against a versioned benchmark and compare results with the approved baseline.

Gate category	Example policy
Critical behavior	No regression allowed on safety or mandatory business rules.
Quality	Aggregate score may not fall below the approved tolerance.
Consistency	Repeated-trial failure rate must remain within policy.
Latency	P95 end-to-end latency may not exceed the service budget.
Resource use	Token or local compute consumption may not increase beyond an accepted threshold.
New failures	Any newly discovered high-severity failure blocks release until reviewed.

TARGET	The user's stated motivation is to bring software-style CI/CD discipline to AI. Automated pipeline enforcement is a logical production target, but the available project description does not confirm a deployed enterprise gate.

19. Observability and Operations

Metric family	Examples
Execution health	Run success rate, evaluator errors, timeouts, queue depth - TARGET.
Quality trend	Pass rate and findings by benchmark version, model, and capability.
RAG diagnostics	Retrieval hit rate, context defects, stage latency.
Agent behavior	Unexpected transitions, tool errors, loop count, termination reason.
Local model health	Load time, tokens per second, memory pressure, failure rate.
Fix effectiveness	Percentage of proposed fixes that pass verification without causing regressions.

Operational dashboards should distinguish test infrastructure failures from target-system failures. A benchmark that did not execute successfully is not a quality failure and should not be compared with a completed baseline run.

Runbook priorities

1. Confirm the test environment and model are available.
2. Validate benchmark and baseline versions.
3. Identify whether failures cluster around one pipeline stage or model route.
4. Reproduce a sample failure outside the batch execution path.
5. Review generated fixes and rerun the full suite before acceptance.

20. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
LangGraph orchestration	Explicit state and adaptive multi-step control.	More complexity than a linear test script.
Three specialized agents	Clear separation of generation, diagnosis, and remediation.	Requires stable contracts between agents.
Local Ollama execution	Zero API cost and stronger data locality.	Performance and model quality depend on local hardware.
Model routing	Match model strength to task requirements.	Routing errors can add variability to evaluation.
ChromaDB	Simple vector-oriented local integration.	Additional structured store needed for mature run governance.
Generated fixes as proposals	Accelerates debugging without bypassing review.	Engineer still owns validation and adoption.
Adaptive testing	Focuses effort on weak areas.	Run-to-run test sets can differ and require careful traceability.

The central trade-off is between adaptive intelligence and reproducibility. AgentProbe gains diagnostic value by changing its tests based on observed weaknesses, but a stable benchmark is still required for reliable baseline comparison. A mature platform should combine both: fixed release gates plus adaptive exploratory testing.

21. Known Limitations

- The available project description does not document a production deployment or organization-wide use.
- Exact benchmark schemas, acceptance thresholds, and scoring formulas are not specified in the source material.
- Local model behavior varies with hardware, model quantization, and runtime configuration.
- Generated fixes can be plausible but ineffective and must be independently verified.
- An evaluator may encode incomplete expectations or judge-model bias.
- ChromaDB alone is not a complete transactional system for benchmark governance and audit history.
- Adaptive tests can make direct run-to-run comparisons harder unless generated cases are versioned.

These limitations do not invalidate the project. They define the boundary between a strong engineering prototype and a production evaluation service. Naming them explicitly improves credibility and guides future work.

22. Modernization Roadmap

Phase	Focus	Deliverables
1. Reproducible core	Stabilize existing workflows.	Versioned cases, model configs, traces, and deterministic report schemas.
2. Measurement	Formalize quality policies.	Metric registry, baseline comparison, repeated-trial analysis.
3. CI integration	Automate pre-deployment checks.	GitHub Actions or CI adapter, blocking and advisory gates.
4. Scale	Support larger suites and teams.	Queue, workers, shared database, object storage, quotas.
5. Production feedback	Learn from real failures.	Convert incidents and user feedback into regression cases.
6. Governance	Make decisions auditable.	Benchmark ownership, approvals, retention, model and prompt lineage.

The recommended sequence avoids premature infrastructure. The platform should first prove that its tests are reproducible and useful. Distributed execution and enterprise governance become valuable only after benchmark quality and developer adoption are established.

23. Lessons Learned

- AI systems need regression engineering, not only prompt experimentation.
- A wrong final answer does not identify the defective layer; stage-level tracing is essential.
- Adaptive tests and fixed benchmarks solve different problems and should coexist.
- Failure verification matters because stochastic output can make one-off anomalies look like defects.
- Automatically generated fixes are useful accelerators but should never bypass benchmark reruns and human review.
- Local models make repeated evaluation affordable and privacy-friendly, but hardware becomes part of the system design.
- Model routing should be observable because model choice influences both quality and reproducibility.
- The value of an evaluation platform is the evidence it produces, not merely a single aggregate score.

24. Capability Matrix and Conclusion

Area	Current documented capability	Production extension
Orchestration	LangGraph-based multi-agent evaluation.	Distributed execution and policy-managed workflows.
Stress testing	Adaptive explore-target-verify cycle.	Benchmark governance and generated-case versioning.
RAG debugging	Seven-stage diagnostic tracing.	Telemetry adapters for multiple RAG frameworks.
Remediation	Generated prompts, chunk configs, and query rewrites.	Approval workflows and automated benchmark verification.
Model execution	Smart router with local Ollama path.	Model registry, quotas, fallbacks, and performance SLOs.
Service layer	FastAPI and React application.	Authentication, RBAC, tenancy, and audit logging.
Storage	ChromaDB in documented stack.	Relational run store and object storage for large artifacts.
Release quality	Pre-deployment testing and benchmarking.	Automated CI/CD regression gates.
Operations	Local project execution.	Central observability, alerting, and production feedback loops.

Conclusion

AgentProbe demonstrates an engineering-first approach to AI quality. Instead of treating a model response as an isolated artifact, the platform evaluates the complete application path: test generation, retrieval behavior, workflow trajectory, output quality, diagnosis, and remediation. Its strongest contribution is the shift from informal prompt checking to repeatable evidence and regression thinking.

The current project establishes the core agentic workflow: adaptive stress testing, seven-stage RAG tracing, fix generation, model routing, local execution, and an API-driven interface. The production roadmap extends those foundations into versioned benchmarks, CI/CD enforcement, distributed execution, governance, and operational feedback.

FINAL PRINCIPLE	An AI change should be treated like a software change: define expected behavior, test it against a controlled baseline, diagnose failures at the correct layer, and prevent regressions from reaching users.

Source Basis and Terminology

This engineering case study is based on the project information supplied in the user's resume and prior project discussions. Verified project terms preserved here include AgentProbe, the Orchestrator Agent, the three-phase explore-target-verify stress test, the seven-step RAG Tracer, the Auto-Fix Agent, the LLM Router, local Ollama execution, and the Python/LangChain/LangGraph/FastAPI/ChromaDB/React stack.

Detailed API routes, database schemas, metric policies, distributed execution, CI enforcement, security governance, and enterprise operations are clearly presented as design specifications or modernization targets because those details were not established as implemented capabilities in the available source material.